

BLOC 4

Maintenir l'application logicielle en condition opérationnelle

Dossier jury - RNCP39583 Expert en développement logiciel

Projet Alcide

Application web de génération d'entraînements et programmes sportifs personnalisés. Candidat : Kevin.

Référence documentaire : 0.13.0-rc.3. État : 28 juillet 2026.

Objet du dossier

Ce dossier présente le monitoring, le traitement des anomalies et la maintenance de l'application. Il distingue les mécanismes versionnés dans le dépôt des preuves d'exécution à joindre avant remise.

Limite de diffusion

Les captures, URLs de runs GitHub, numéros d'issues et retours utilisateur ne sont renseignés que lorsqu'ils existent réellement. Toute mise en situation est identifiée explicitement comme une simulation déclarée.

Livrable écrit individuel - 20 pages maximum

1. Exigences et périmètre MCO

Le Bloc 4 évalue la maintenance d'un logiciel développé pendant la formation : mises à jour, supervision, consignation et correction des anomalies, évolutions, versions et collaboration avec le support.

Compétence	Attendu évalué	Réponse Alcide
C4.1.1	Mettre à jour les dépendances de manière sécurisée.	Dependabot, audit, CI et procédure de qualification/rollback.
C4.1.2*	Supervision, indicateurs, sondes, seuils et signalement.	Readiness API, healthcheck Web, monitoring GitHub horaire, artefact et issue automatique.
C4.2.1*	Collecter, consigner et analyser une anomalie.	Processus d'incident, templates GitHub et fiches BUG documentées.
C4.2.2	Corriger et déployer via l'intégration et le déploiement continu.	PR checklist, CI/CD, smoke tests et rollback Vercel documentés.
C4.3.2*	Établir un journal des versions et des correctifs.	CHANGELOG versionné, matrice de traçabilité maintenance.
C4.3.3	Collaborer avec le support sur un problème complexe.	Template support et cas pilote à présenter comme réel ou simulation déclarée.

* Compétence éliminatoire selon le règlement de certification.

Architecture de production

Composant	Rôle	Point de maintenance
Web Next.js	Interface utilisateur et OAuth	Healthcheck /api/health, erreurs d'authentification, version.
API Hono	Logique métier, IA, accès aux données	Liveness /health et readiness /health/ready.
Neon PostgreSQL	Persistance et migrations Drizzle	Disponibilité, erreurs de connexion, branche/backup avant migration.
Vercel + GitHub Actions	Déploiement et contrôles	CI, CD, smoke tests, monitoring et rollback.

2. Mises à jour des dépendances

La maintenance des dépendances couvre les packages npm et les GitHub Actions. Dependabot est configuré pour ouvrir des propositions hebdomadaires ; le lockfile assure la reproductibilité. Une mise à jour n'est intégrée qu'après qualification, contrôles automatisés et validation ciblée.

Étape	Action	Preuve / critère de sortie
1. Détecter	Consulter Dependabot ou lancer pnpm outdated et pnpm audit.	Mise à jour qualifiée.
2. Évaluer	Identifier patch, minor, major, sécurité, runtime ou build.	Impact et rollback définis.
3. Isoler	Traiter dans une branche dédiée avec le lockfile.	Changement traçable.
4. Vérifier	Lint, types, tests, couverture, build et audit CI.	CI verte et aucune vulnérabilité non justifiée.
5. Déployer	Fusionner puis exécuter CD et smoke tests si nécessaire.	Service stable ou rollback.
6. Tracer	Mettre à jour le changelog pour un changement notable.	Version/Unreleased cohérent.

Rythme retenu

Type	Fréquence	Règle
Sécurité critique	Sous 24 h si exploitable	Priorité maximale, validation renforcée et rollback prêt.
Patch/minor	Hebdomadaire	Traitement après CI verte.
Major framework	Mensuelle ou lot planifié	Changelog éditeur, tests ciblés et plan de retour.
Migration DB	À chaque besoin	Branche/backup Neon avant application sensible.

Preuve à joindre

Capter une PR Dependabot ou un audit de dépendances effectivement traité. La configuration seule ne doit pas être décrite comme une mise à jour déjà exécutée.

3. Supervision et alerte

Le système versionné surveille les interfaces publiques critiques. Le workflow GitHub Actions Monitoring - Production health est planifié à la minute 17 de chaque heure et peut être lancé manuellement.

Sonde	Contrat contrôlé	Finalité
API readiness	GET /health/ready ; HTTP 200 et JSON status: ready.	Vérifier que l'API se déclare prête à servir des requêtes.
Web health	GET /api/health ; HTTP 2xx et JSON status: ok.	Vérifier la disponibilité du point de santé du Web.
CD	Smoke tests après déploiement.	Détecter une régression immédiatement après livraison.
CI	Lint, types, tests, build et audit.	Prévenir la livraison d'un changement non conforme.

Paramètres réellement configurés

Paramètre	Valeur configurée	Interprétation
Cadence	17 * * * *	Une exécution horaire à :17 UTC.
Seuil HTTP	200 <= status < 300	Toute réponse non 2xx est en échec.
Contrat JSON	ready pour API ; ok pour Web	Évite de considérer un simple 200 comme suffisant.
Délai/reprise	20 s par tentative ; 2 retries ; 5 s	Tolère une instabilité brève, puis remonte l'échec.
Rapport	Artefact production-health-report, même en échec	Conserve horodatage, URLs, codes et payloads.

Les endpoints de santé sont non cacheables. Ce monitoring ne mesure pas encore les métriques métier détaillées (taux 5xx, p95 IA, coût IA) : elles sont traitées comme axes d'amélioration, non comme contrôles déjà actifs.

4. Signalement et preuve C4.1.2

Lorsqu'une sonde échoue, le workflow publie une issue GitHub Production healthcheck failed avec le label monitoring, ou commente l'issue déjà ouverte. Lors d'un passage ultérieur réussi, il commente puis ferme cette issue. Le canal de signalement déclaré est donc GitHub Actions -> artefact -> GitHub Issue.

Élément	Statut	Ce qui doit être montré au jury
Workflow et issue automatique	Configurés dans le dépôt	Le YAML et le protocole de supervision.
Run de production réussi	À joindre	Capture de la date, du job vert et des deux contrôles.
Artefact de monitoring	À joindre	Rapport production-health-report du même run.
Alerte test sûre	Disponible à exécuter	Cycle simulate_alert puis simulate_recovery, sans requête sur la production.

Élément	Statut	Ce qui doit être montré au jury
Notification humaine	Dépend des réglages GitHub	Ne la revendiquer que si une capture prouve la réception.

Démonstration sans panne de production

- Dans Actions, lancer Monitoring - Production health avec `check_mode = simulate_alert`.
- Capturer le job rouge, l'artefact et l'issue [TEST] Production healthcheck alert simulation avec le label `monitoring-test`.
- Relancer avec `check_mode = simulate_recovery` et capturer la fermeture de cette issue test.
- Présenter la simulation comme telle. Elle vérifie le circuit GitHub sans provoquer d'indisponibilité ni modifier l'application de production.

Décision de présentation

Better Stack ou un outil équivalent reste un renfort optionnel. Il ne doit être cité comme actif que si ses moniteurs, seuils et destinataire sont réellement configurés et capturés.

5. Collecte et consignation des anomalies

Une anomalie peut provenir du monitoring, d'un test, de la CI/CD, des logs, d'un audit ou d'un retour utilisateur. Le processus est commun : détecter, qualifier, reproduire, analyser, prioriser, corriger ou contourner, valider, informer puis clôturer.

Étape	Informations requises	Sortie
Détection	Source, date, environnement, composant, symptômes.	Signal ou ticket créé.
Qualification	Criticité P0-P3, impact, fréquence, utilisateur concerné.	Priorité et responsable.
Reproduction	Étapes, données de test, résultat attendu/observé, logs utiles.	Bug reproductible ou statut À reproduire.
Analyse	Cause racine/hypothèse et options de correction/contournement.	Décision technique argumentée.
Validation	Tests, CI, smoke test et lien correctif/PR si disponibles.	Statut résolu ou suivi planifié.

Outil de collecte

Le formulaire GitHub Anomalie Bloc 4 impose la source, l'environnement, le composant, la criticité, la description, les étapes de reproduction, l'impact, la cause, le correctif et la validation. Les fiches BUG-001 et BUG-002 conservent également l'historique de cas rencontrés pendant le projet.

L'annexe C4.2.1 fournit un modèle prêt à compléter. Un ticket fictif est possible dans la simulation RNCP, mais doit comporter la mention Simulation déclarée et ne doit jamais être présenté comme un incident réel.

6. Exemple de traitement : BUG-002

BUG-002 traite un README encodé en UTF-16 LE, rendant son affichage GitHub illisible. Ce cas illustre une anomalie de documentation réellement identifiée pendant le projet, sans revendiquer un incident de production.

Rubrique	Constat et traitement
Impact	README difficile à lire dans GitHub, mauvaise compréhension du projet et des consignes de maintenance.
Détection	Revue finale : caractères parasites et octets nuls visibles entre les caractères.
Reproduction	Lire le fichier dans un outil affichant l'encodage ou inspecter les premiers octets.
Cause	Création/copie Windows ayant conservé un encodage UTF-16 au lieu d'UTF-8.
Correctif	Réécriture en UTF-8 sans BOM et recommandation de règles .gitattributes.
Validation	Contrôle des octets et lecture GitHub/terminal attendue sans caractères parasites.
Prévention	Vérifier l'encodage des documents avant commit et utiliser un format texte UTF-8.

Passage CI/CD

Pour tout correctif applicatif, la checklist de PR impose lint, typecheck, tests, couverture, build, healthchecks si nécessaire, mise à jour du changelog et documentation du rollback. La CI est le garde-fou avant déploiement ; le smoke test production n'est revendiqué que lorsqu'il est joint.

7. Journal des versions et traçabilité

Le CHANGELOG suit Keep a Changelog et utilise les rubriques Added, Changed, Fixed et Security. La version documentaire de référence est 0.13.0-rc.3. Elle est une candidate : elle ne doit pas être appelée version de production sans preuve de déploiement et smoke tests associés.

Version	Date	Éléments maintenus
0.13.0-rc.3	20/07/2026	Correctifs de recette authentifiée ; validation de production à joindre séparément.
0.13.0-rc.2	20/07/2026	Readiness API, tests, monitoring GitHub, templates Bloc 4 et healthchecks non cacheables.
0.12.0	16/04/2026	Pagination, dashboard, statistiques et correction Auth.js trustHost.

Chaîne obligatoire pour chaque maintenance notable

- Ticket anomalie ou cas support, avec statut réel ou simulation déclarée.
- Décision de correction, contournement ou rollback, puis PR/commit lorsqu'il existe.
- Validation technique : commande, CI, test manuel et test de production seulement s'ils ont été exécutés.
- Entrée CHANGELOG et version/Unreleased correspondant au statut réellement livré.

La matrice C4.3.2 en annexe relie ces éléments et empêche de confondre un brouillon, un correctif fusionné, une validation locale et une livraison de production.

8. Support client et continuité de service

Le formulaire Cas support client Bloc 4 structure le canal, le contexte, le problème, le diagnostic, la contribution technique, les rôles des parties prenantes, la résolution et la validation. Aucune donnée personnelle ne doit être déposée.

Cas	Traitement à présenter	Règle de transparence
Retour réel	Conserver le ticket daté, les informations anonymisées, la résolution et le retour de validation.	Ne pas divulguer de données sensibles.
Absence de retour réel	Utiliser le modèle support pour une génération IA lente ou en échec, avec diagnostic et contournement.	Titre et contenu marqués Simulation déclarée.

Rollback et reprise

Situation	Réponse prévue	Preuve utile
Régression Web/API	Promouvoir le dernier déploiement Vercel sain ou appliquer un hotfix après qualification.	Capture déploiement sain/rollback et smoke test.
Migration DB risquée	Préparer une branche ou sauvegarde Neon avant migration ; privilégier des migrations compatibles.	Capture branche/backup avant exécution.
Incident de disponibilité	Consulter le rapport de monitoring, qualifier l'impact, corriger/contourner, puis valider la reprise.	Issue, artefact et test de rétablissement.

Le rollback Vercel est documenté. Le rollback base de données reste une stratégie à prouver avant une migration sensible ; il ne doit pas être décrit comme déjà exécuté sans pièce datée.

9. Recommandations priorisées

Priorité	Recommandation	Gain / effort
Haute	Joindre un run vert, son artefact et le cycle d'alerte test GitHub.	Sécurise la démonstration C4.1.2 ; effort faible.
Haute	Créer une issue anomalie complète et un cas support réel ou déclaré simulé.	Renforce C4.2.1/C4.3.3 ; effort faible.
Haute	Capturer Dependabot/audit et, avant migration, branche ou backup Neon.	Renforce la maintenance préventive et le rollback.
Moyenne	Centraliser les logs avec un logger structuré et des alertes 5xx/DB/IA.	Diagnostic plus rapide ; 1 à 2 jours estimés.
Moyenne	Ajouter une observabilité IA : latence, erreurs, coût et quotas.	Pilotage de la dépendance fournisseur ; 1 à 2 jours.
Moyenne	Ajouter des tests d'intégration DB dédiés.	Couvre repositories et migrations ; 2 à 3 jours.

Positionnement jury

La supervision GitHub est présentée comme un dispositif configuré et contrôlable. Les améliorations d'observabilité avancée sont des recommandations réalistes, pas des fonctions déjà livrées.

10. Checklist de remise et conclusion

Pièces à joindre avant remise

- Run vert Monitoring - Production health, avec les deux contrôles et l'artefact production-health-report.
- Démonstration simulate_alert puis simulate_recovery, clairement identifiée comme test sûr et non comme panne réelle.
- Une anomalie consignée et un cas support réel ou explicitement simulé, reliés à la matrice de traçabilité.
- Run CI final vert de la révision remise et preuve de déploiement uniquement si la candidate est annoncée en production.
- Cohérence entre package.json, guide de déploiement, CHANGELOG et ce dossier : 0.13.0-rc.3 est une candidate documentaire.

Conclusion

Alcide possède une base MCO versionnée et crédible : dépendances surveillées, CI/CD, readiness, healthchecks non cacheables, monitoring horaire, rapport d'exécution, circuit d'issue, processus d'anomalies, correctifs documentés, changelog et rollback Vercel.

Les trois compétences éliminatoires sont adressées par des mécanismes concrets. La remise est sécurisée lorsque les captures d'exécution et les tickets requis complètent ces mécanismes sans transformer une configuration ou une simulation en fait de production.

Annexes de preuve

Références à joindre selon les faits disponibles : protocole C4.1.2, modèle anomalie C4.2.1, matrice C4.3.2, modèle support C4.3.3, BUG-001/BUG-002, CHANGELOG et preuves GitHub datées.